

CHAPTER 2, PART 3

Lists

Key topics

- Debugging in NetBean IDE
- Single Linked List: Implementation
 - Non-Generic SingleLinkedList
 - Single linked list of values of primitive type
 - Private Node class

Debug Programs using Netbean IDE

- Debugging process (basics)
 1. **Set breakpoints:** click line number bar to the left of the desired line where you want the breakpoint to be
 2. **Start the debugger:** right-click the program file name in the project list pane. click **Debug file**
 3. Click **Step Over**  to run code line by line
 4. Click **Step Into**  to enter the called method
 5. Click **Continue**  to run multiple lines from current breakpoint to the next breakpoint
 6. While at a breakpoint, check if current variables are correct. (Check **Variables tab** at the bottom of GUI)
 7. Click  to end the debugging

Implementing Single Linked List of values of primitive type like `int`

- Do not use generics in implementation
- Node storing a value of primitive type like `int`
 - **Public** Node class (package `linkedlist_int_node_public`,
FOR SELF-READING, NOT REQUIRED)
 - `Node.java` – Node definition
 - `UseNodes.java` – How to directly connect Node objects
 - `SimpleLinkedList.java`
 - Definition of single linked list of `int` values
 - `SimpleLinkedListTest.java`
 - A Java application that uses the class `SimpleLinkedList`.

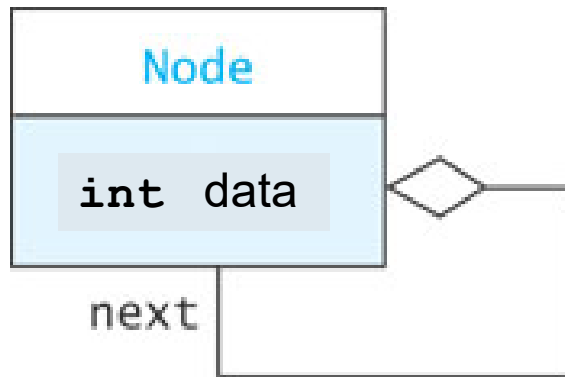
Implementing Single Linked List of values of primitive type like `int`

- **Private** Node class (package `linkedlist_int_node_private`)
 - ▣ Has same Node structure as public Node
 - ▣ But SimpleLinkedList class can directly access data fields in Node class using dot operator like
 - `current.data, current.next`
 - ▣ SimpleLinkedList.java
 - Definition of single linked list of `int` values
 - Contains the definition of a private Node class inside
 - ▣ SimpleLinkedListTest.java
 - A Java application that uses the class SimpleLinkedList.

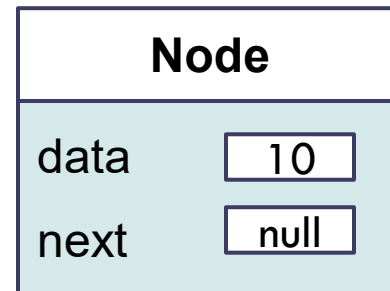
Structure in a Node object for SimpleLinkedList class

- A **Node** object in a single linked list of `int` values contains:
 - ▣ a **data item of `int` type**
 - ▣ one **link** referencing the Node immediately after

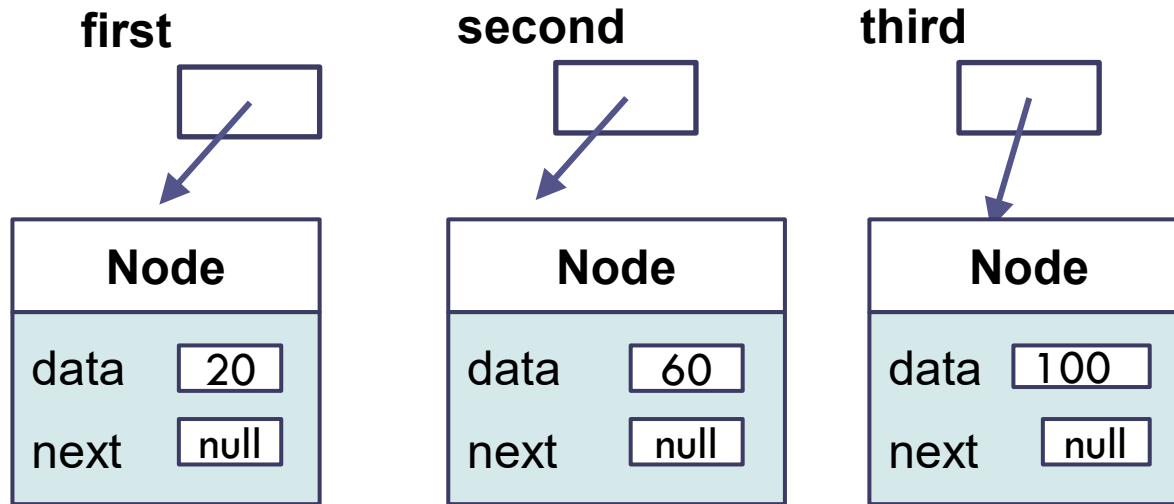
Node structure:



Sample Node object



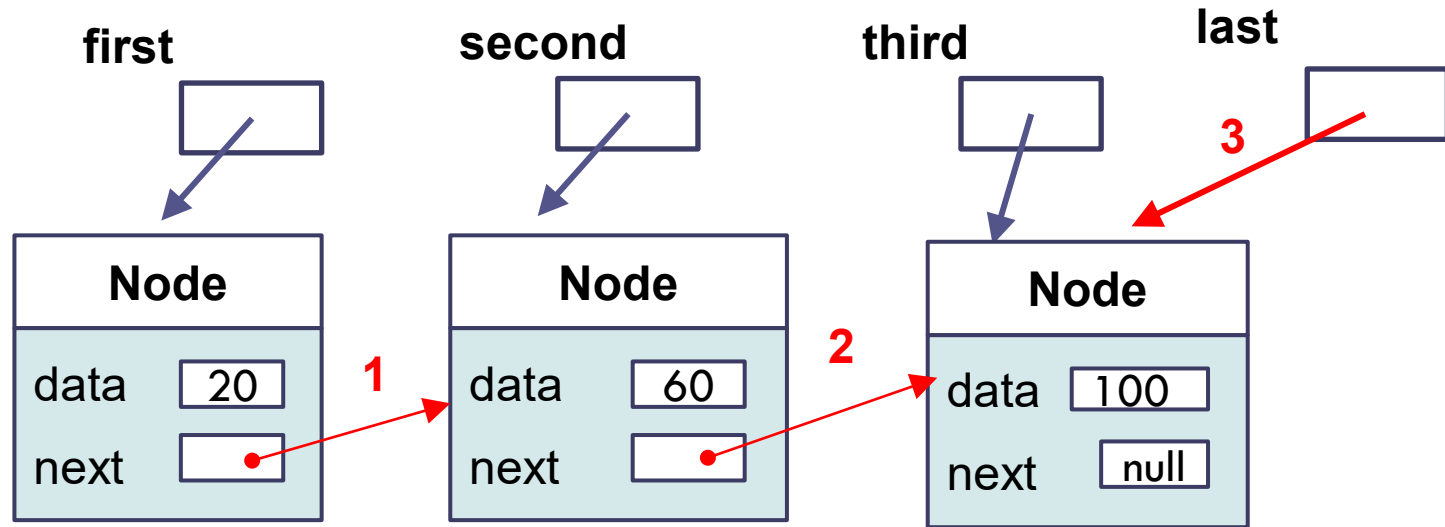
Creating Node objects Before Forming a Single Linked List of `int` values



`SimpleLinkedList.main():`

```
Node first = new Node(20);
Node second = new Node(60);
Node third = new Node(100);
```

Directly Connect Node objects and Form a Single Linked List of `int` values



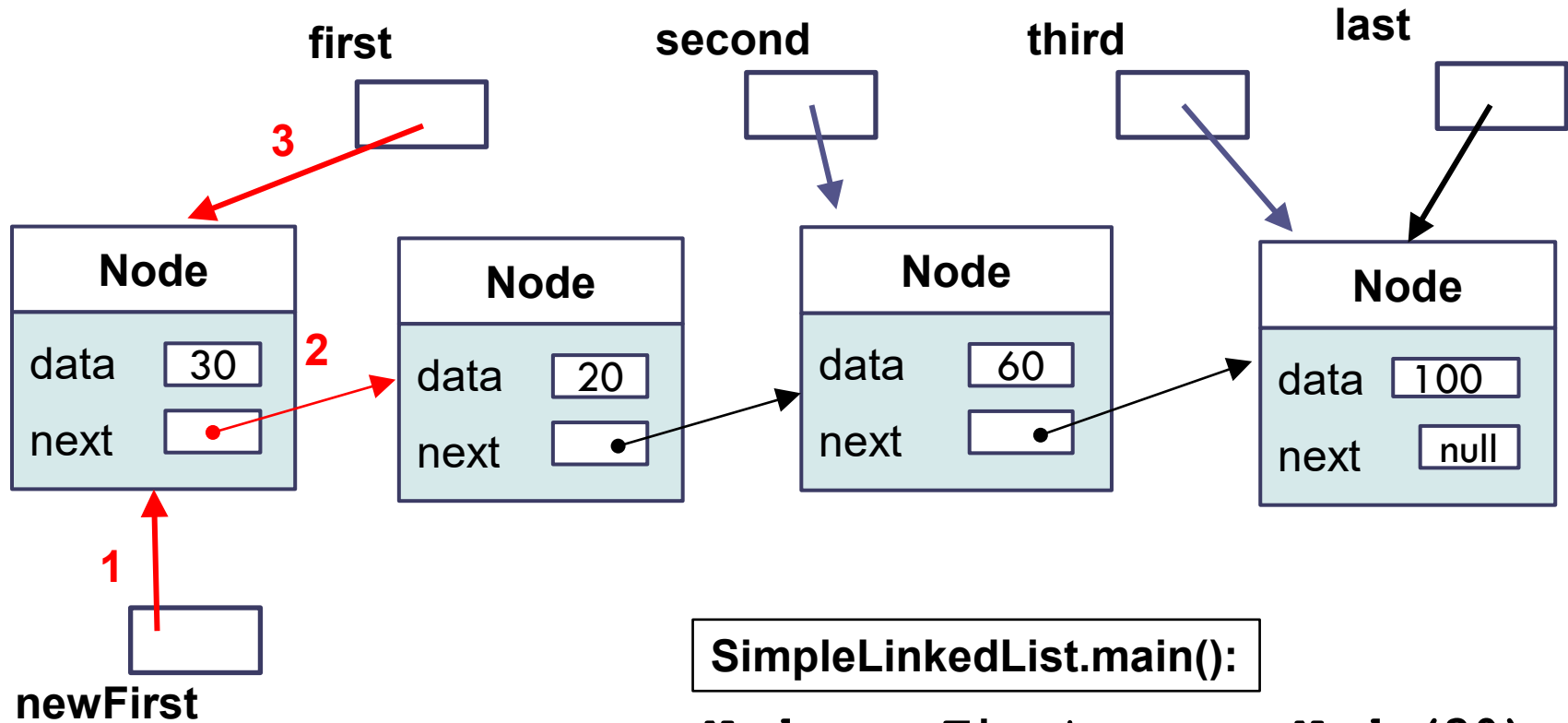
```
SimpleLinkedList.main():
```

```
first.next = second;
```

```
second.next = third;
```

```
Node last = third;
```

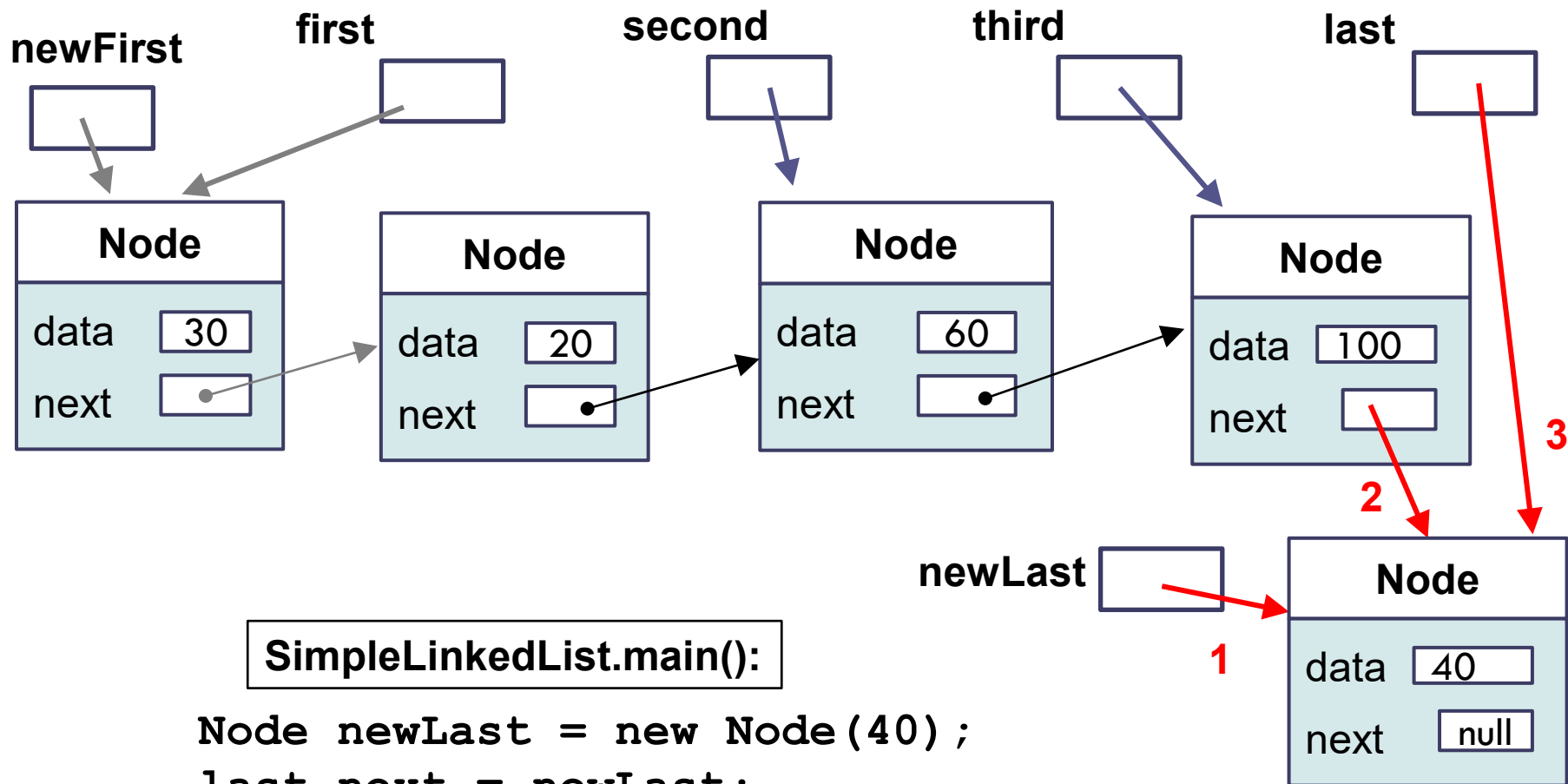

Add a Node to the Front of the Single Linked List of `int` values



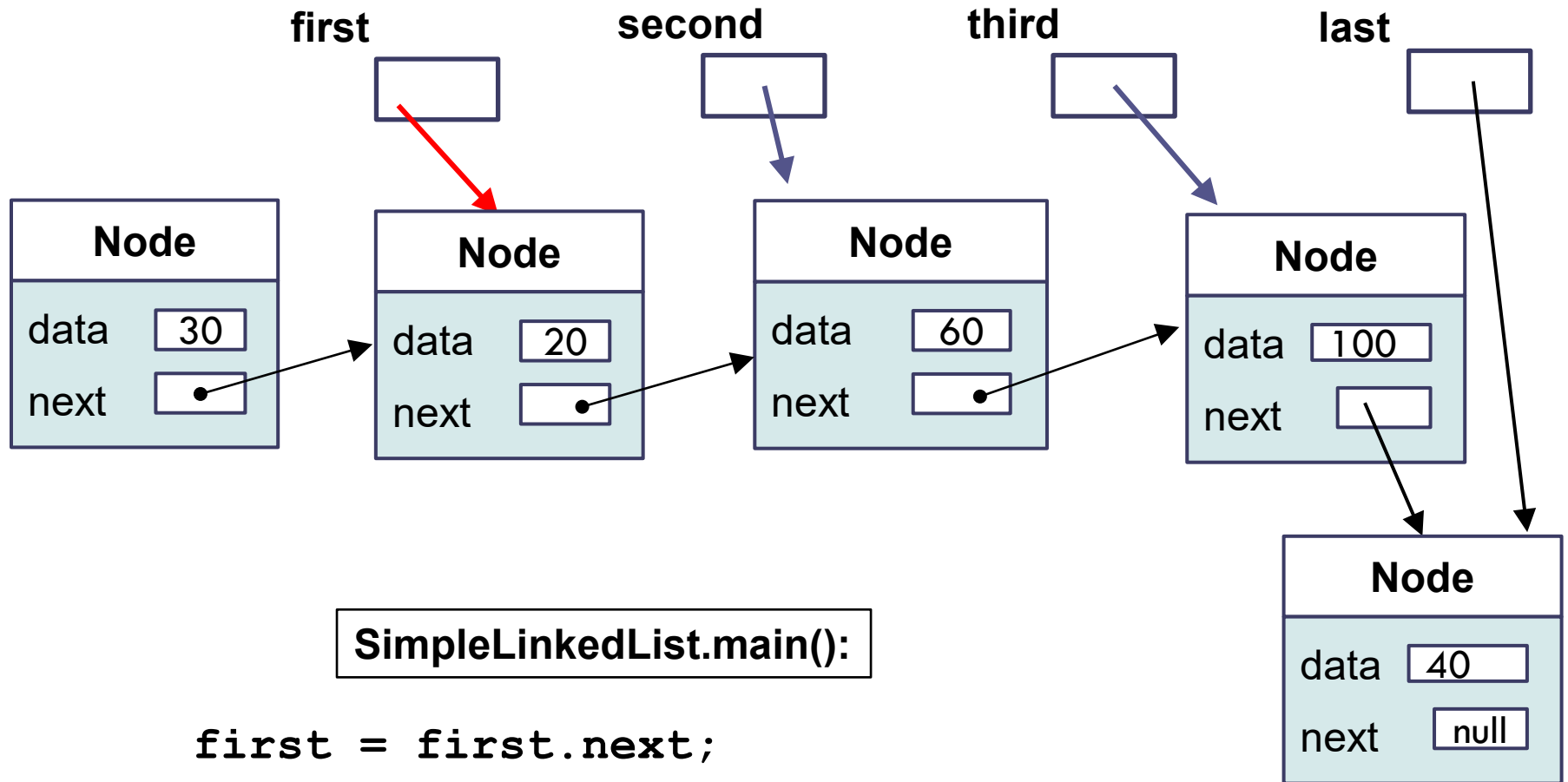
```
SimpleLinkedList.main():
```

```
Node newFirst = new Node(30);
newFirst.next = first;
first = newFirst;
```

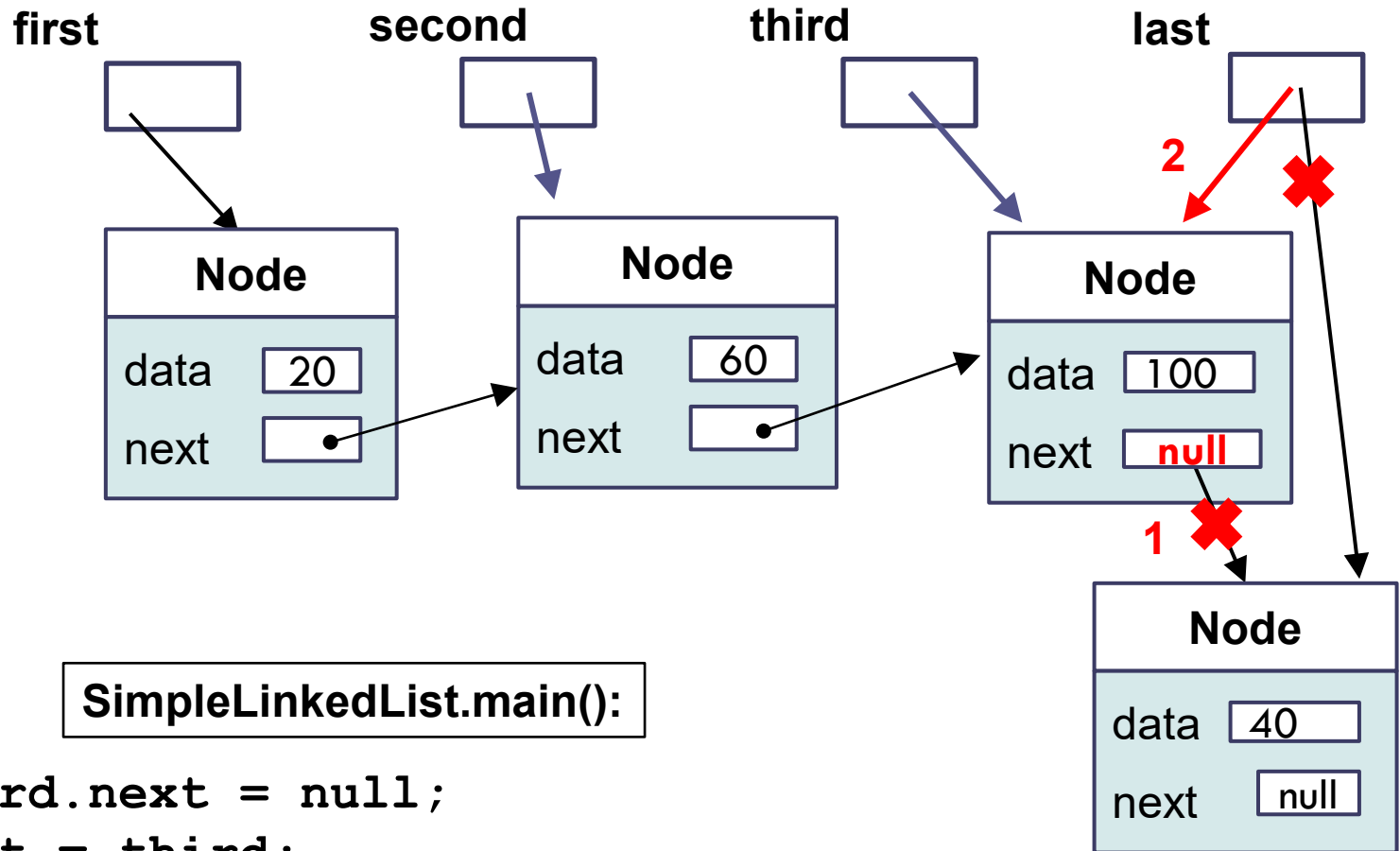
Add a Node to the End of the Single Linked List of `int` values



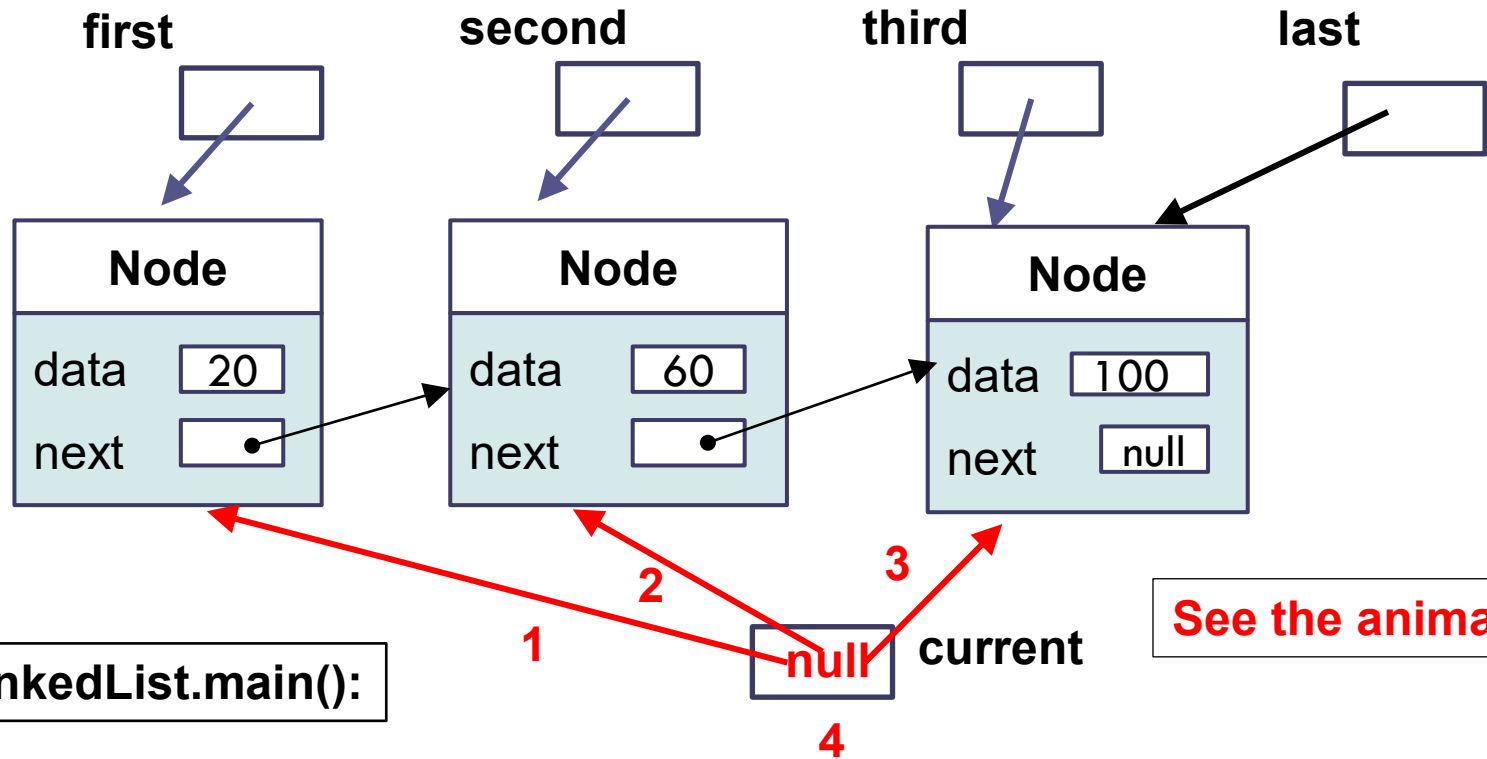
Remove First Node of the Single Linked List of `int` values



Remove Last Node of the Single Linked List of `int` values



Traverse Node objects in a Single Linked List of `int` values



`SimpleLinkedList.main():`

```
Node current = first;
while (current != null) {
    System.out.println(current.data);
    current = current.next;
}
```

An Example of Single Linked List of `int` values using class `SimpleLinkedList`

`SimpleLinkedListTest.java`:

